Straight-path cost for the FL controller: joint-line cost vs. Jacobian cost

WP1 – `Controllers/FL.py`, `Final_Code/path_constraint_jacobian.py`

In one line. Both methods add a quadratic path term $x_t^\top Q_t^{\text{path}} x_t$ to the same FL LQG problem. **Yours** penalises the distance to a *straight line in joint space*, mostly in the last 50 ms. **The Jacobian cost** penalises the hand's *actual distance to the straight hand line*, linearised with $J(\theta)$, at every step. Same simulation, same weights, only Q_t^{path} changes.

1 Common setting: the FL controller

State (8): angles, velocities, target angles, start angles

$$x = [\theta_s,\ \theta_e,\ \omega_s,\ \omega_e,\ \theta_s^{\text{tg}},\ \theta_e^{\text{tg}},\ \theta_s^0,\ \theta_e^0]^\top.$$

Feedback linearisation turns the arm into a double integrator, $\ddot{\theta} = v$, so with $\Delta t = 10$ ms, $N = 60$ steps:

$$x_{t+1} = A\,x_t + B\,v_t, \qquad A = \mathbb{I}_8 + \Delta t\,(e_1 e_3^\top + e_2 e_4^\top), \qquad B = \Delta t\,(e_3 e_1^\top + e_4 e_2^\top).$$

Cost (weights of `common.py`: $w_p = 2 \cdot 10^4$, $w_v = 10$, $r = 6 \cdot 10^{-5}$):

$$\mathcal{J} = \underbrace{w_p \|\theta_N - \theta^{\text{tg}}\|^2 + w_v \|\omega_N\|^2}_{x_N^\top Q_f\, x_N} + \sum_{t=0}^{N-1} \Big(\underbrace{x_t^\top Q_t^{\text{path}} x_t}_{\text{path term}} + r\,\|v_t\|^2 \Big).$$

Riccati ($S_N = Q_f$, backwards in t) and **command**:

$$L_t = (rI + B^\top S_{t+1} B)^{-1} B^\top S_{t+1} A, \qquad S_t = Q_t^{\text{path}} + A^\top S_{t+1}(A - BL_t), \qquad v_t = -L_t\,\hat{x}_t,$$

$$u_t = \big(\mathcal{A}\,\text{diag}(f_l f_v)\big)^+ \big(M(\theta)\,v_t + C(\theta,\omega) + B_{\text{visc}}\,\omega\big),$$

with $\hat{x}_t$ the Kalman estimate (60 ms delay), $\mathcal{A}$ the moment arms, f_l, f_v the force–length and force–velocity gains. *Everything here is shared*; the two methods only differ in Q_t^{path}.

2 Your method: joint-line cost (`compute_path`)

1. A line in joint space. Take the hand point 75% of the way, $p^{\text{ref}} = p_0 + 0.75\,(p_f - p_0)$, its posture $\theta^{\text{ref}} = \text{IK}(p^{\text{ref}})$, and the slope

$$k = \frac{\theta_e^{\text{ref}} - \theta_e^0}{\theta_s^{\text{ref}} - \theta_s^0}.$$

2. Distance to that line (linear in x, uses the start angles stored in the state):

$$d_J(x) = k\,(\theta_s - \theta_s^0) - (\theta_e - \theta_e^0) = h^\top x, \qquad h = [\,k,\ -1,\ 0,\ 0,\ 0,\ 0,\ -k,\ 1\,]^\top.$$

3. Path term, end-loaded with $\tau = 0.02$ s:

$$\boxed{Q_t^{\text{path}} = w_c\, e^{-(N-1-t)\Delta t/\tau}\, h\,h^\top \quad \Rightarrow \quad x_t^\top Q_t^{\text{path}} x_t = w_c\, e^{-(N-1-t)\Delta t/\tau}\, d_J(x_t)^2}$$

Each step back from the end divides the weight by $e^{0.5} \approx 1.65$: the last 5 steps (50 ms) carry 92% of it.

Consequences.

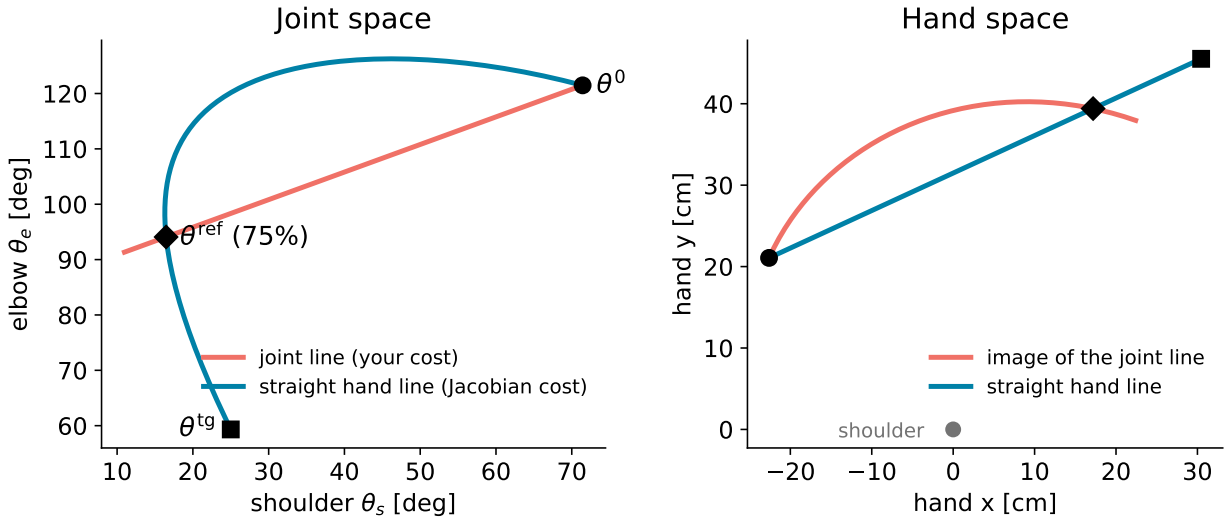


Figure 1: Movement 1. Your joint line (through θ^0 and θ^{ref}) is straight in joint space but curved for the hand, and it does not pass through the target θ^{tg}. The straight hand line is curved in joint space. $k = 0.50$.

- Exact quadratic, no approximation, very cheap.
- But a *proxy*: a straight joint line is not a straight hand path (Fig. 1, right).
- The line misses the target, and the cost acts at the end: it pulls against reaching θ^{tg} at rest $\Rightarrow$ late correction, higher terminal and peak velocities.
- Straightening stops around $w_c \approx 80$–100 (1.8–2.7 cm); above, the path curves again.

3 Jacobian method: quadratised hand deviation

1. The quantity we actually want small: the lateral distance of the hand to the line.

$$p(\theta) = \begin{bmatrix} l_1 \cos\theta_s + l_2 \cos(\theta_s + \theta_e) \\ l_1 \sin\theta_s + l_2 \sin(\theta_s + \theta_e) \end{bmatrix}, \qquad a = \frac{p_f - p_0}{\|p_f - p_0\|}, \quad n = \begin{bmatrix} -a_y \\ a_x \end{bmatrix}, \qquad \boxed{e(\theta) = n^\top (p(\theta) - p_0)}$$

e is *not* quadratic in $\theta \Rightarrow$ linearise it.

2. Linearise at a posture on the line. Pick θ_t^* with $p(\theta_t^*)$ on the line, so $e(\theta_t^*) = 0$:

$$e(\theta) \;\approx\; n^\top J(\theta_t^*)\,(\theta - \theta_t^*) = g_t^\top \theta - c_t, \qquad g_t = J(\theta_t^*)^\top n, \quad c_t = g_t^\top \theta_t^*,$$

$$J(\theta) = \begin{bmatrix} -l_1 \sin\theta_s - l_2 \sin(\theta_s + \theta_e) & -l_2 \sin(\theta_s + \theta_e) \\ l_1 \cos\theta_s + l_2 \cos(\theta_s + \theta_e) & l_2 \cos(\theta_s + \theta_e) \end{bmatrix}.$$

The error is second order in $\theta - \theta_t^*$, and the cost itself keeps θ close to the line.

3. Make it quadratic: add a constant state. $g_t^\top \theta - c_t$ is affine, so use $z = [x;\, 1]$ (9 states):

$$\boxed{Q_t^{\text{path}} = w_J\, h_t h_t^\top, \qquad h_t = [\,g_t^\top,\; 0_{1\times 6},\; -c_t\,]^\top \qquad \Rightarrow \qquad z_t^\top Q_t^{\text{path}} z_t = w_J\,(g_t^\top \theta_t - c_t)^2}$$

$$A_z = \begin{bmatrix} A & 0 \\ 0 & 1 \end{bmatrix}, \quad B_z = \begin{bmatrix} B \\ 0 \end{bmatrix}, \quad \text{same Riccati} \;\Rightarrow\; v_t = -L_t \begin{bmatrix} \hat{x}_t \\ 1 \end{bmatrix} = -L_t^x \hat{x}_t - \ell_t \quad \text{(affine feedback)}.$$

Uniform in time: every step is penalised equally.

4. Where to linearise (θ_t^*). Start from a minimum-jerk progress along the line,

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \qquad \theta_t^* = \text{IK}(p_0 + s(t/N)\,(p_f - p_0)),$$

then optionally re-linearise: run the noiseless controller, project its hand path onto the line, recompute θ_t^* (3 times). Here it barely matters: the linearised deviation is within 5% of the real one even 12 cm off the line, and exact to < 0.05 cm once straight.

4 Side by side

	Joint-line (yours)	Jacobian
Penalised quantity	$d_J = k(\theta_s - \theta_s^0) - (\theta_e - \theta_e^0)$	$e \approx g_t^\top \theta - c_t$, hand distance to the line
Space	joint (proxy)	hand (the real target)
Exact?	exact quadratic of a proxy	first-order approximation of the real quantity
When	last $\sim$50 ms ($e^{-\text{time to go}/\tau}$)	every step, uniform
Matrix	fixed $h\,h^\top$	time-varying $h_t h_t^\top$
State	8	9 (constant 1, affine feedback)
Needs	IK of one point	IK along the line, a reference timing
Weight range used	$w_c \in \{0, \ldots, 80\}$ (rad^2)	$w_J \in \{0, \ldots, 0.01\}$ (cm^2)

5 Results (two 58 cm movements, 600 ms)

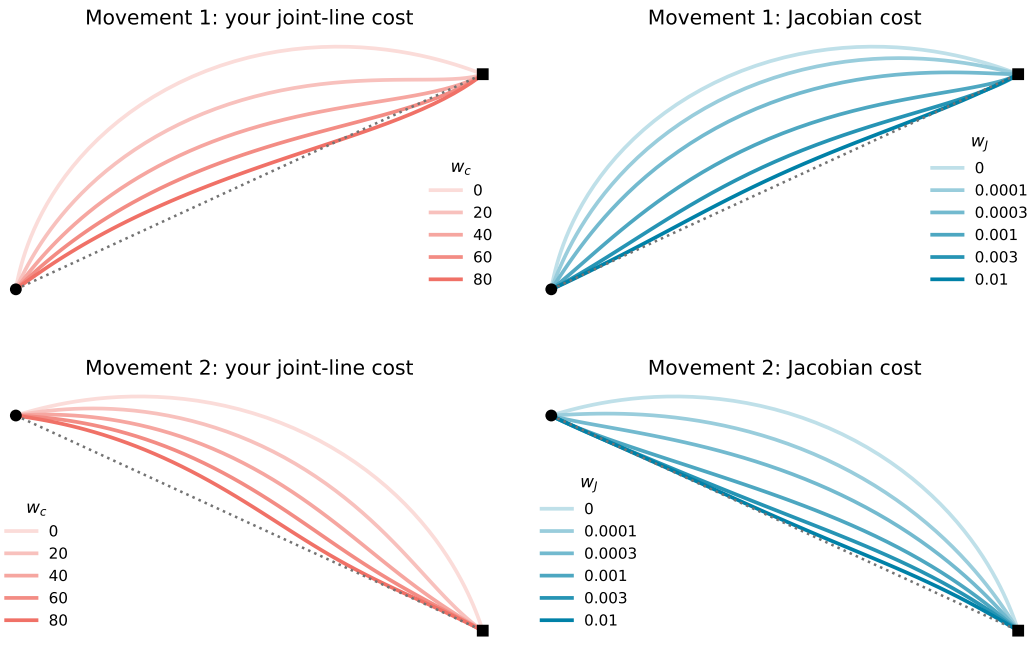


Figure 2: Noiseless hand paths over each weight sweep (light → dark = more weight). Jacobian gains with 3 re-linearisations; without them the paths differ by less than 0.6 cm.

At matched straightness (200 noisy trials each; your straightest setting $w_c = 80$):

	lateral dev.	terminal vel.	peak vel.	effort $\sum u^2$	movement cost
Mvt 1, $w_c = 80$	1.8 cm	12.0 °/s	346 °/s	32.2	5.06
Mvt 1, $w_J = 0.003$	2.6 cm	**4.2** °/s	**227** °/s	32.7	**3.79**
Mvt 1, $w_J = 0.01$	**1.0** cm	**3.8** °/s	241 °/s	35.6	4.05
Mvt 2, $w_c = 80$	2.7 cm	6.7 °/s	234 °/s	59.0	6.77
Mvt 2, $w_J = 0.003$	**1.6** cm	**4.0** °/s	263 °/s	**34.7**	**3.96**
No path cost (mvt 1 / 2)	13.6 / 12.0 cm	~3.9 °/s	157 / 254 °/s	26.7 / 55.2	3.19 / 6.02

- **Straighter:** down to 1.0 / 0.75 cm noisy, < 0.1 cm noiseless (yours stops at 1.8 / 2.7 cm).
- **Stops at the target:** terminal velocity at the noise floor (~4 °/s, as with no path cost). Endpoint error is the same for all (~0.15–0.20 cm).
- **Smoother** on mvt 1 (peak 227 vs 346 °/s), **41% less effort** on mvt 2.

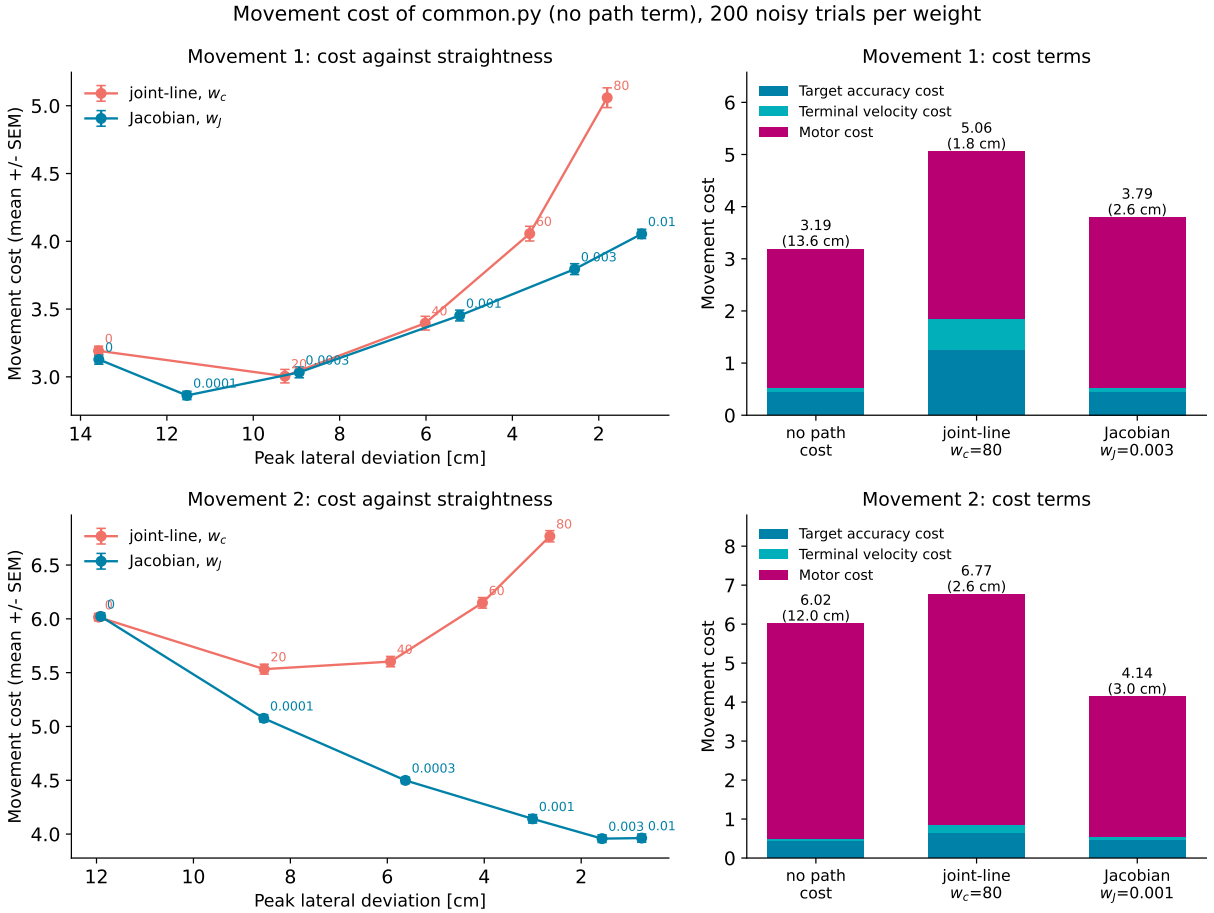


Figure 3: Movement cost of `common.py`: $w_p\|\theta_N - \theta^{\text{tg}}\|^2 + w_v\|\omega_N\|^2 + 0.1\sum_t \|u_t\|^2$ (muscle commands), without the path term, which differs between methods.

Movement cost.

- Yours makes straight paths expensive: +59% (mvt 1) and +12% (mvt 2) over no path cost. The end-loaded term fights the arrival: accuracy cost ×2.8, terminal velocity cost ×7.7 on mvt 1.
- Jacobian keeps accuracy and terminal velocity at their no-path level; only the motor term moves: +19% in total for mvt 1 (at 2.6 cm), **−31**% to **−34**% for mvt 2 (at 3.0 to 1.6 cm), where the straight path is *cheaper* than the FL default path.

Simulation time (one core, per movement; per-trial runs are identical):

	Joint-line	Jacobian
Gains, once per weight	1.8 ms	3.6 ms (no re-linearisation) 83 ms (3 re-linearisations)
One noisy trial (60 steps, Kalman with delay)	17.5 ms	17.5 ms
200 trials, one weight	3.50 s	3.50 s / 3.58 s (+2%)

The per-step Kalman filter dominates; the Riccati recursion is under 10%. For reference, `run_fl` (used by `path_constraint.py`) recomputes the gains on every trial: 18.8 ms per trial.

6 Limitations of the Jacobian method

- **First order only.** It minimises the *linearised* deviation. Small here ($\leq 5\%$), but it would grow for highly curved maps or very weak weights.
- **Needs a reference timing θ_t^*.** Gains are computed offline: a large perturbation that delays or advances the hand (e.g. a force field) moves the linearisation point. Not tested.
- **The whole line must be reachable** and away from singularities ($\det J \neq 0$). Here the elbow stays within 59°–126°; a line near the shoulder or at full extension fails.
- **One extra state** (the constant 1): `simulate_FL` is written for 8 states, so the script uses its own copy of the loop (it reproduces `simulate_FL` exactly).
- **Weights not comparable** with w_c (cm^2 vs rad^2): compare at equal straightness.